Project Design Document

Subtitle: A collaborative file sharing system.

Document Metadata

Field	Value
Project Name	Synapse
Author	Ashok Bavireddy
Status	Draft / In-Review / Approved
Created Date	Apr 6, 2026
Last Updated	Apr 6, 2026
Version	v1.0

1. Overview

Synapse is a real-time collaborative platform designed as a website to enable seamless synchronization and instant resource exchange within any group

environment. In settings where collective coordination is essential, traditional communication tools often involve complex setups that slow down the sharing process. Synapse simplifies this by allowing for the creation of instant, temporary groups, enabling participants to connect and share information the moment it is needed. Built as a unified digital workspace, it ensures all participants remain aligned regardless of the group's scale or location. The project's ultimate goal is to provide a secure, high-speed environment that facilitates efficient collaboration and immediate connectivity.

2. Problem Statement & Goals

2.1 Problem Statement

When ad-hoc groups—whether students, colleagues, or event attendees—need to collaborate and share files temporarily, existing platforms create unnecessary friction. Traditional file-sharing requires a slow 'upload-then-share-link' workflow and often relies on third-party cloud storage. There is a need for a lightweight platform where users can instantly spin up a temporary workspace, share resources directly, and collaborate with maximum speed and minimum setup.

2.2 Project Goals

- **Instant Session Creation:** Allow authenticated users to quickly create and join secure, temporary session rooms using a generated access code to facilitate immediate collaboration.
- **Real-Time Data Streaming:** Implement WebSockets to facilitate instantaneous live chat and presence tracking (join/leave notifications) without the latency of traditional HTTP polling.
- **Optimized File Sharing:** Stream binary file data in real-time chunks, allowing participants to receive files quickly while bypassing the friction of third-party cloud storage links.
- **Bandwidth Efficiency:** Utilize a custom server-side interceptor to prevent the "echoing" of large file chunks back to the original sender, drastically reducing server load and bandwidth consumption.

2.3 Non-Goals

- **Persistent Data Storage:** Synapse is designed for live, ephemeral collaboration. It is not a cloud storage alternative (like Google Drive). Once a session is terminated by the host, chat history and file streams are not permanently archived in a database.
- **Audio and Video Conferencing:** While the platform handles real-time text and binary file transfer, it will not support live audio or video calls (e.g., WebRTC media streams) to maintain a lightweight server architecture.
- **Massive File Hosting (e.g., 2GB+):** The system is optimized for the rapid sharing of standard project resources, documents, and media. It is not built to act as a torrenting or raw large-scale data transfer tool.
- **Complex Role Management:** The platform relies on a simple Host/Participant dynamic for the live sessions. It will not include complex organizational structures, permissions, or multi-tiered admin roles.

3. Technical Summary: Synapse Core Design

3.1 Core Identity

Synapse is a web-based, ephemeral, real-time collaboration engine. It enables instant group workspaces (Sessions) for high-speed file sharing and chat, designed to be used by logged-in users. Data is not permanently persisted, ensuring a clean slate upon session termination.

3.2 Problem & Solution

- **Problem:** Latency and privacy friction in ad-hoc file sharing (requires accounts/links).
- **Solution:** A P2P-inspired WebSocket relay that streams binary data in chunks directly between connected clients via a central "smart" broker.

3.3 Technical Stack

- **Backend:** Java 17, Spring Boot 3.x, Spring Messaging (STOMP/SockJS).
- **Frontend:** React.js, Stomp.js.
- **State Management:** In-memory ConcurrentHashMap (no database I/O).

- **Security:** JWT-based session handshakes + temporary 6-digit access codes.

3.4 High-Level Architecture & Data Flow

- **Session Lifecycle (REST):** POST /create (Trainer) and POST /join (Participant) manage the initial handshake.
- **Real-Time Relay (WebSockets):** Uses a Pub/Sub model on /topic/session/{id}/file-stream and /chat.
- **Binary Chunking Engine:** Files are fragmented into **16KB chunks**, encoded as Base64 strings, and reassembled as Blob objects on the client side to bypass browser RAM limits.

3.5 Key Optimization: Sender-Bypass Interceptor

- **Mechanism:** A custom ChannelInterceptor (preSend) caches the senderSocketId.
- **Logic:** Before broadcasting to the room topic, the interceptor compares the senderSocketId with the recipientSocketId.
- **Outcome:** If identical, the packet is dropped. This eliminates "echo" bandwidth waste, effectively doubling the sender's upload efficiency.

3.6 Constraints (Non-Goals)

- **Persistence:** Zero disk storage; all data is wiped upon session termination.
- **Media:** No WebRTC (A/V) support; focused strictly on text and binary data.
- **Scale:** Optimized for standard office/training resources (not designed for 2GB+ multi-part transfers).

4. Implementation Strategy

4.1 Development Phases

Phase 1: Environment Setup & Core Messaging Architecture

- Initialize the Spring Boot backend with Websocket and Security dependencies.

- Configure the STOMP over WebSocket message broker and define core endpoints.
- Set up the React frontend using Vite and establish the initial connection to the backend using SockJS and Stomp.js.
- Implement basic "Join/Leave" presence tracking to verify successful handshake.

Phase 2: Feature Integration & Stream Optimization

- Develop the Live Chat component using WebSocket-only routing (removing REST dependencies).
- Build the Binary Streaming Engine on the frontend to fragment files into 16KB chunks.
- Implement the Sender-Bypass Interceptor on the backend to filter echoed data.
- Configure server-side WebSocket transport buffers to handle the high throughput of simultaneous streams.

Phase 3: Validation, Refinement & Final Deployment

- Conduct concurrency testing to ensure the in-memory ConcurrentHashMap handles multiple sessions without collisions.
- Perform memory profiling on the browser to verify chunk reassembly does not cause RAM spikes.
- Deploy the integrated application to a cloud environment (e.g., AWS Elastic Beanstalk) that supports long-lived TCP connections.
- Finalize documentation and "Synapse" branding for the course presentation.

4.2 Resource Requirements

Resource Type	Description
Hardware	**Developer Machine:** Intel i3 11th Gen / 8GB RAM (Local Dev). **Production Server:** AWS EC2 (t3.medium recommended for WebSocket buffer memory).

Resource Type	Description
Software	**Backend:** JDK 17+, Maven/Gradle. **Frontend:** Node.js (v18+), VS Code. **Utilities:** Postman (WS Testing), Chrome DevTools (Memory Profiling).
Human Capital	**Backend Engineering:** Expertise in Spring WebSocket & Interceptors. **Frontend Engineering:** React Hook architecture & File API knowledge. **DevOps:** Knowledge of Cloud deployment & WebSocket buffering.

5. Security & Privacy Considerations

Synapse is built on the principle of **Data Minimization**. By eliminating account registration and personal data collection, the system inherently avoids the storage of Personally Identifiable Information (PII). All session data is stored exclusively in volatile **In-Memory** structures, ensuring that once a session is terminated, no residual digital footprint remains on the server.

Security Risk	Mitigation Strategy (Actually Coded)
Data Interception	Implementation of **SockJS and STOMP** protocols designed for secure handshaking over encrypted channels.
Cross-Site Scripting (XSS)	React's **Automatic Escaping** is utilized in the chat UI; messages are rendered as plain text within tags, preventing the execution of malicious scripts.
Unauthorized Topic Access	**JWT (JSON Web Tokens)** are required for the initial HTTP handshake; the WebSocket connection is only upgraded after the token is validated.

Security Risk	Mitigation Strategy (Actually Coded)
Bandwidth/Buffer Attacks	**WebSocket Transport Registration** is configured with strict limits (512KB message size, 5MB buffer) to prevent memory exhaustion from oversized streams.
Bandwidth Sniffing/Waste	A custom **SenderBypassInterceptor** is implemented to identify the source socket and drop packets meant for the original sender, securing the stream from unnecessary "echo" loops.
Information Persistence	The **Ephemeral Logic** in the SessionService ensures that liveSessions.remove(sessionId) is called upon termination, instantly clearing all memory references for the Garbage Collector.
Session Hijacking	Use of **UUID-based Session IDs** and unique **6-digit join codes** ensures that rooms cannot be easily guessed or accessed by unauthorized participants.

6. Open Questions & Risks

Risk 1: Browser Heap Exhaustion during File Reassembly The current implementation reassembles 16KB Base64 chunks into a single array before creating a Blob. For very large files, the overhead of Base64 (33% increase) and the memory required to hold the full reassembled string can exceed the browser's heap limit, potentially crashing the user's tab.

- **Mitigation Plan:** Implement a maximum file size cap (e.g., 200MB) in the handleFileUpload logic and investigate the use of the **FileSystemWritableFileStream API** to stream chunks directly to the disk instead of RAM.

Risk 2: Lack of Delivery Acknowledgement (Backpressure) In the current broadcast model, the server "blindly" sends chunks to all participants. If a student has a slow internet connection, the server’s outbound buffer may overflow,

leading to a forced disconnection of that student without an automated way to resume the missed chunks.

- **Mitigation Plan:** Implement a basic "Chunk Acknowledgement" signal or a "Resume from Index" request that allows a client to re-sync if the WebSocket connection blips.

Risk 3: Single-Server Scalability (Vertical Bound) Since all active sessions are stored in an in-memory ConcurrentHashMap in the SessionService, the project is vertically bound to a single server instance. If the server restarts or if we attempt to load balance across multiple servers, session state will be lost or fragmented.

- **Mitigation Plan:** For future scaling, transition the SessionService to use a **Redis Pub/Sub** or a distributed cache to synchronize session states across multiple backend nodes.

Question 1: Audit vs. Privacy Policy Since the project is built as an ephemeral workspace where all data is wiped upon termination, how should we handle "Reporting" if a participant shares malicious or inappropriate content during a live session? Should we implement a "Host-Only Log" that persists for 24 hours?

Question 2: Technical Feasibility of WebRTC Integration Given the 2GB file limitation discussed, is it feasible to implement a "Hybrid" module where the WebSocket handles the signaling (handshake), but the browser automatically switches to a **WebRTC DataChannel** for files exceeding 100MB to bypass the server relay entirely?

Question 3: Mobile Network Stability How does the current chunking logic perform on mobile devices when switching between Wi-Fi and 5G? Should we implement an "Aggressive Reconnect" strategy in the useSessionRoom hook to prevent session loss during network handovers?

7. References

- GitHub Link: https://github.com/notashock/mini-project-hcl
- Deployed Link: mini-project-hcl.vercel.app